BÀI THỰC HÀNH CTDL

BẢNG BĂM (HASH TABLE) VÀ TỪ ĐIỂN (DICTIONARY)

Cấu trúc dữ liệu Bảng băm cho phép chúng ta truy xuất dữ liệu gần như tức thì chỉ bằng cách sử dụng một "khóa" duy nhất, giống như bạn tra một từ trong cuốn từ điển để tìm nghĩa của nó vậy.

Lưu ý về sự khác biệt so với C#: C# phân biệt hai kiểu — `Hashtable` (cũ, non-generic, phải ép kiểu khi lấy ra) và `Dictionary<TKey, TValue>` (hiện đại, generic, an toàn kiểu). Python chỉ có MỘT cấu trúc duy nhất là `dict` — nhưng vì Python định kiểu động, một `dict` của Python vừa "generic tự nhiên" (không cần khai báo kiểu key/value) giống Dictionary, vừa tiềm ẩn rủi ro lưu trữ dữ liệu không đồng nhất giống Hashtable, vì Python không kiểm tra kiểu tại thời điểm biên dịch. Ở Bài tập 3, chúng ta vẫn sẽ mô phỏng lại đúng tình huống và rủi ro của Hashtable bằng một `dict` lưu dữ liệu không đồng nhất, để hiểu rõ vì sao kiểm tra kiểu (type-checking) vẫn quan trọng trong Python.

Mục Đích và Yêu Cầu Chung

Mục đích

- **Hiểu rõ mô hình Key-Value:** Nắm vững cách lưu trữ và truy xuất dữ liệu thông qua một khóa duy nhất.
- **Thành thạo các thao tác CRUD:** Thực hiện nhuần nhuyễn các hoạt động Thêm, Đọc, Cập nhật, Xóa trên `dict`.
- **Hiểu rõ đặc điểm của `dict` trong Python:** Nắm được ưu điểm (an toàn, linh hoạt, hiệu năng) và những rủi ro khi lưu dữ liệu không đồng nhất (tương tự tinh thần của Hashtable trong C#).
- **Giải quyết bài toán thực tế:** Áp dụng `dict` để giải quyết các vấn đề phổ biến như đếm tần suất, lập chỉ mục, caching...

Yêu cầu chung cho tất cả bài tập

- **Môi trường:** Viết ứng dụng dạng **Console Application** bằng **Python 3** (chạy bằng lệnh `python ten_file.py` trong terminal).
- **Code sạch:** Viết code rõ ràng, dễ đọc, tuân theo chuẩn **PEP 8**. Sử dụng type hint (vd. `dict[str, int]`) khi kiểu dữ liệu đã rõ ràng để tăng khả năng đọc hiểu.
- **Kiểm tra sự tồn tại:** Luôn sử dụng toán tử `in` (vd. `if key in my_dict:`) hoặc phương thức `.get(key)` để kiểm tra một khóa đã tồn tại hay chưa trước khi thao tác, tránh lỗi `KeyError`.
- **Học sâu với AI:** Tận dụng AI để khám phá các khái niệm nền tảng như "hàm băm (hash function)", "xung đột băm (hash collision)" và các kịch bản sử dụng nâng cao.

BÀI TẬP 1: Đếm tần suất xuất hiện của các từ — *Mức độ: Dễ (dict)*

Đây là bài tập kinh điển để làm quen với `dict`, giúp bạn hiểu rõ logic "kiểm tra-cập nhật hoặc thêm mới".

Yêu cầu chi tiết

1. Viết một chương trình nhận một câu văn bản từ người dùng.
2. Sử dụng `dict[str, int]` để đếm số lần xuất hiện của mỗi từ trong câu.
3. Key là từ (str), Value là số lần xuất hiện (int).

4. Chương trình không phân biệt chữ hoa/thường (ví dụ: "The" và "the" được coi là một).
5. Cuối cùng, in ra tất cả các từ cùng với tần suất của chúng.

Gợi ý

- Dùng `text.split()` để tách câu thành các từ (mặc định tách theo khoảng trắng, tự bỏ khoảng trắng thừa).
- Với mỗi từ, hãy chuyển nó về dạng chữ thường bằng `.lower()`.
- Dùng `if word in word_frequencies:` để kiểm tra từ đã có trong dict chưa.
- Nếu có, tăng giá trị của nó lên 1: `word_frequencies[word] += 1`.
- Nếu chưa, thêm từ đó vào với giá trị ban đầu là 1: `word_frequencies[word] = 1`.
- Nâng cao (không bắt buộc): có thể thay toàn bộ logic đếm bằng `collections.Counter(words)` — nhưng nên tự cài đặt tay trước để hiểu rõ bản chất.

Code mẫu

```
def main():
    text = input("Nhập một câu để đếm tần suất từ: ").lower()

    # Loại bỏ các dấu câu đơn giản
    punctuation = ".,?!"
    for p in punctuation:
        text = text.replace(p, "")

    words = text.split()
    word_frequencies: dict[str, int] = {}

    for word in words:
        if word in word_frequencies:
            word_frequencies[word] += 1
        else:
            word_frequencies[word] = 1

    print("\n--- Tần suất xuất hiện của các từ ---")
    for word, count in word_frequencies.items():
        print(f"Từ '{word}': {count} lần")


if __name__ == "__main__":
    main()
```

Vận dụng AI để gia tăng trải nghiệm

- **Hỏi về lý thuyết:** Hỏi AI: "Hàm băm (hash function) hoạt động như thế nào bên trong một dict của Python? Tại sao nó giúp việc tìm kiếm nhanh hơn nhiều so với list?"
- **Khám phá thêm:** Hỏi AI: "Làm thế nào để sắp xếp kết quả đầu ra của chương trình này theo tần suất giảm dần?" (AI sẽ hướng dẫn bạn dùng `sorted(word_frequencies.items(), key=lambda x: x[1], reverse=True)`).

BÀI TẬP 2: Quản lý thông tin sinh viên — *Mức độ: Dễ - Trung bình (dict)*

Bài tập này nâng cao hơn bằng cách sử dụng một đối tượng tùy chỉnh làm Value, một kịch bản rất phổ biến.

Yêu cầu chi tiết

1. Tạo một lớp `Student` với các thuộc tính: `full_name` (str), `date_of_birth` (date), `average_score` (float) — có thể dùng `@dataclass` cho gọn.
2. Sử dụng `dict[str, Student]` để quản lý danh sách sinh viên. Key là **Mã số sinh viên** (MSSV - str).
3. Xây dựng menu chức năng: 1. Thêm sinh viên mới · 2. Tìm sinh viên theo MSSV · 3. Xóa sinh viên theo MSSV · 4. Hiển thị toàn bộ danh sách · 5. Thoát.

Gợi ý

- Định nghĩa class `Student` trước (khuyến khích dùng `@dataclass` để tự sinh `__init__`).
- Khi thêm sinh viên, hãy kiểm tra MSSV đã tồn tại chưa bằng `if mssv in student_database:`.
- Để tìm sinh viên, dùng `student_database.get(mssv)` sẽ an toàn hơn là dùng `student_database[mssv]` trực tiếp, vì nó trả về `None` thay vì ném lỗi `KeyError` nếu không tìm thấy key.
- Để xóa, dùng `del student_database[mssv]` (sau khi đã kiểm tra tồn tại) hoặc `student_database.pop(mssv, None)`.

Code mẫu

```
from dataclasses import dataclass
from datetime import date

@dataclass
class Student:
    full_name: str
    date_of_birth: date
    average_score: float

    def __str__(self) -> str:
        return f"Tên: {self.full_name}"


student_database: dict[str, Student] = {}

# Phần menu tương tự các bài trước, đây là logic cốt lõi

# 2. Tìm sinh viên
mssv_to_find = input("Nhập MSSV cần tìm: ")
found_student = student_database.get(mssv_to_find)
if found_student is not None:
    print(f"Tìm thấy sinh viên: {found_student}")
else:
    print("Không tìm thấy sinh viên với MSSV này.")

# 3. Xóa sinh viên
mssv_to_remove = input("Nhập MSSV cần xóa: ")
if mssv_to_remove in student_database:
```

```
    del student_database[mssv_to_remove]
    print("Đã xóa sinh viên thành công.")
else:
    print("Không tìm thấy sinh viên để xóa.")
```

Vận dụng AI để gia tăng trải nghiệm

- **Tích hợp kỹ năng khác:** Hỏi AI: "Làm thế nào để tôi có thể dùng list comprehension hoặc hàm `filter()` để tìm tất cả sinh viên có điểm trung bình trên 8.0 từ dict này?" (AI sẽ gợi ý `[s for s in student_database.values() if s.average_score > 8.0]`).
- **Tư duy thiết kế:** Hỏi AI: "Trong trường hợp nào thì nên dùng MSSV (str) làm key, và trong trường hợp nào thì nên dùng một số ID (int) làm key? Phân tích ưu nhược điểm."

BÀI TẬP 3: Làm việc với dữ liệu không đồng nhất — *Mức độ: Trung bình (dict "kiểu Hashtable")*

Bài tập này buộc bạn phải lưu trữ dữ liệu không đồng nhất trong một `dict` để hiểu rõ sự "linh hoạt" và cả những rủi ro tương tự Hashtable trong C#.

Yêu cầu chi tiết

1. Giả sử bạn đang nhận dữ liệu từ một hệ thống cũ, nơi thông tin của một sản phẩm được lưu trữ trong một `dict` không kiểm tra kiểu chặt chẽ.
2. Tạo một `dict` và thêm vào đó các thông tin sau cho một sản phẩm: key "ID" → 101 (int) · key "Name" → "Laptop Pro" (str) · key "Price" → 2500.50 (float) · key "IsAvailable" → True (bool).
3. Viết chương trình lấy các giá trị này ra từ `dict` và in chúng ra màn hình theo một định dạng đẹp.
4. Thử gán lại key "ID" bằng một giá trị khác và quan sát: khác với `Hashtable.Add()` trong C# (sẽ ném lỗi khi trùng khóa), gán trực tiếp `product_info["ID"] = ...` trong Python sẽ **ghi đè êm thấm, không báo lỗi**. Hãy tự viết thêm logic để mô phỏng lại hành vi "báo lỗi khi trùng khóa" giống Hashtable.Add().

Gợi ý

- Python không yêu cầu ép kiểu (cast) khi lấy dữ liệu ra như C# — đây chính là điểm khác biệt lớn nhất so với Hashtable.
- Nhưng vì không có kiểm tra kiểu tại thời điểm biên dịch, bạn **nên** tự kiểm tra kiểu bằng `isinstance()` trước khi dùng dữ liệu theo cách yêu cầu đúng kiểu (vd. định dạng số, phép toán số học) để tránh lỗi `TypeError` khi chạy.
- Để mô phỏng hành vi "lỗi khi trùng khóa" của `Hashtable.Add()`, viết một hàm `add_unique(d, key, value)` tự kiểm tra `if key in d: raise KeyError(...)` trước khi gán.
- Để cập nhật giá trị bình thường (ghi đè có chủ đích), chỉ cần gán lại: `product_info["Price"] = 2450.00`.

Code mẫu

```
def add_unique(d: dict, key, value) -> None:
    """Mô phỏng Hashtable.Add(): báo lỗi nếu key đã tồn tại."""
    if key in d:
        raise KeyError(f"Khóa '{key}' đã tồn tại trong dict.")
    d[key] = value
```

```
def main():
    product_info = {}

    # Thêm dữ liệu (mô phỏng Hashtable.Add, báo lỗi nếu trùng khóa)
    add_unique(product_info, "ID", 101)
    add_unique(product_info, "Name", "Laptop Pro")
    add_unique(product_info, "Price", 2500.50)
    add_unique(product_info, "IsAvailable", True)

    try:
        id_ = product_info["ID"]
        name = product_info["Name"]
        price = product_info["Price"]
        is_available = product_info["IsAvailable"]

        # Không cần ép kiểu như C#, nhưng nên kiểm tra kiểu nếu dữ liệu
        # đến từ nguồn không đáng tin cậy
        if not isinstance(price, (int, float)):
            raise TypeError("Price phải là kiểu số.")

        print("--- Thông tin sản phẩm ---")
        print(f"ID: {id_}")
        print(f"Tên: {name}")
        print(f"Giá: {price:,.2f} VNĐ")
        print(f"Tình trạng: {'Còn hàng' if is_available else 'Hết hàng'}")
    except (KeyError, TypeError) as ex:
        print("Lỗi khi xử lý dữ liệu:", ex)

    # Thử thêm lại key 'ID' bằng add_unique -> sẽ báo lỗi (giống Hashtable.Add)
    try:
        add_unique(product_info, "ID", 202)
    except KeyError as ex:
        print("Lỗi khi thêm trùng khóa:", ex)

    # Cập nhật giá trị hợp lệ: gán trực tiếp (Python luôn cho phép ghi đè)
    product_info["Price"] = 2450.00
    print(f"\nGiá mới sau khi cập nhật: {product_info['Price']:,.2f} VNĐ")


if __name__ == "__main__":
    main()
```

💬 Vận dụng AI để gia tăng trải nghiệm

- **So sánh trực tiếp:** Hỏi AI: "Vì Python chỉ có một loại dict, hãy liệt kê 3 rủi ro lớn nhất khi dùng dict để lưu dữ liệu không đồng nhất (giống tinh thần của Hashtable trong C#), so với việc định nghĩa một class hoặc dataclass rõ ràng."
- **Xử lý lỗi:** Cố tình gây lỗi kiểu trong code (ví dụ: `int(product_info["Name"])`). Copy lỗi và hỏi AI: "Lỗi ValueError/TypeError này có nghĩa là gì? Làm thế nào để viết code Python an toàn hơn để phòng tránh nó khi làm việc với dict chứa dữ liệu không đồng nhất?" (AI sẽ gợi ý dùng `isinstance()` hoặc thư viện `pydantic` để validate dữ liệu).

BÀI TẬP 4: Mô phỏng hệ thống Cache đơn giản — *Mức độ: Khó (dict)*

Bài tập này giới thiệu một ứng dụng cực kỳ quan trọng của `dict` trong thực tế: làm bộ nhớ đệm (cache) để tăng tốc độ ứng dụng.

Yêu cầu chi tiết

1. Giả sử bạn có một hàm `get_data_from_database(query)` mô phỏng việc lấy dữ liệu từ database, và hàm này rất tốn thời gian (dùng `time.sleep()` để giả lập).
2. Tạo một `dict[str, str]` để làm cache. Key là câu query (str), Value là kết quả trả về.
3. Viết một hàm `get_smart_data(query)` hoạt động như sau: kiểm tra xem query có trong cache không → nếu có (cache hit), lấy và trả về kết quả trực tiếp từ cache, in ra thông báo "Lấy từ Cache" → nếu không có (cache miss), gọi `get_data_from_database(query)`, lưu kết quả vào cache, rồi mới trả về, in ra thông báo "Lấy từ Database".
4. Chạy thử với cùng một query nhiều lần để thấy sự khác biệt về thời gian.

Gợi ý

- `_cache` cần được khai báo ở phạm vi module (bên ngoài hàm) để nó có thể tồn tại qua nhiều lần gọi hàm.
- Hàm `get_data_from_database` chỉ đơn giản là `time.sleep(3)` (dừng 3 giây) và trả về một dữ liệu giả lập.
- Logic chính nằm trong việc kiểm tra `query in _cache`.
- Nâng cao (không bắt buộc): Python có sẵn decorator `functools.lru_cache` giúp cache tự động kết quả hàm — hãy thử dùng nó sau khi đã tự cài đặt tay để so sánh.

Code mẫu

```
import time

# Cache được khai báo ở cấp module để tồn tại lâu dài
_cache: dict[str, str] = {}


def get_data_from_database(query: str) -> str:
    """Giả lập hàm lấy dữ liệu chậm."""
    print(f"...Đang thực hiện truy vấn '{query}' đến database (chậm)...")
    time.sleep(3)  # Giả lập độ trễ 3 giây
    return f"Kết quả cho '{query}'"


def get_smart_data(query: str) -> str:
    if query in _cache:
        print("[CACHE HIT] Lấy dữ liệu từ Cache.")
        return _cache[query]

    print("[CACHE MISS] Không có trong Cache, đang lấy từ Database.")
    result = get_data_from_database(query)
    _cache[query] = result  # Lưu vào cache cho lần sau
    return result
```

```
if __name__ == "__main__":
    print("Lần 1:", get_smart_data("SELECT * FROM Users"))
    print("-" * 15)
    print("Lần 2:", get_smart_data("SELECT * FROM Products"))
    print("-" * 15)
    print("Lần 3:", get_smart_data("SELECT * FROM Users"))  # Lần này sẽ nhanh
```

Vận dụng AI để gia tăng trải nghiệm

- **Tư duy hệ thống:** Hỏi AI: "Trong một ứng dụng thực tế, cache có thể lớn dần lên và chiếm hết bộ nhớ. Hãy gợi ý một vài chiến lược để giới hạn kích thước cache (cache eviction policies) như LRU hoặc FIFO, và cách cài đặt LRU cache đơn giản trong Python."
- **Nâng cao:** Hỏi AI: "Hệ thống cache dùng dict thường trong bài có an toàn khi chạy trong môi trường đa luồng (multi-threading) ở Python không? Nếu không, tôi cần dùng cơ chế nào (`threading.Lock`, hay cấu trúc dữ liệu nào) để thay thế?"

BÀI TẬP 5: Xây dựng chỉ mục ngược (Inverted Index) — *Mức độ: Rất khó (dict)*

Đây là bài tập nâng cao, mô phỏng nguyên lý cốt lõi của các cỗ máy tìm kiếm. Bạn sẽ xây dựng một cấu trúc dữ liệu cho phép tìm kiếm nhanh văn bản.

Yêu cầu chi tiết

1. Cho một tập hợp các "tài liệu" (mỗi tài liệu là một chuỗi):

```
documents = [
    "the quick brown fox",        # Document 0
    "jumps over the lazy dog",    # Document 1
    "the quick brown dog",        # Document 2
]
```

2. Xây dựng một "chỉ mục ngược" bằng `dict[str, list[int]]`. **Key** là một từ (str). **Value** là một danh sách (`list[int]`) chứa ID của các tài liệu (chỉ số trong mảng) có chứa từ đó.
3. Sau khi xây dựng xong chỉ mục, viết chức năng cho phép người dùng nhập một từ và chương trình sẽ in ra danh sách các tài liệu chứa từ đó.
4. Ví dụ kết quả của chỉ mục: "the" → [0, 1, 2], "quick" → [0, 2], "dog" → [1, 2], "lazy" → [1].

Gợi ý

- Bạn cần một vòng lặp lồng: vòng ngoài duyệt qua các tài liệu (dùng `enumerate(documents)` để có cả chỉ số `i`), vòng trong duyệt qua các từ trong tài liệu đó.
- Với mỗi từ, kiểm tra xem nó đã có trong `inverted_index` chưa.
- Nếu chưa, tạo một `list[int]` rỗng mới cho từ đó rồi thêm chỉ số tài liệu `i` vào.
- Nếu đã có, lấy `list[int]` tương ứng ra, thêm chỉ số `i` vào (nhớ kiểm tra để không thêm trùng).
- Nâng cao (không bắt buộc): có thể dùng `collections.defaultdict(list)` để bỏ bớt bước kiểm tra "đã có key chưa".

Code mẫu

```
def main():
```

```
    documents = [
        "the quick brown fox",
        "jumps over the lazy dog",
        "the quick brown dog",
    ]

    inverted_index: dict[str, list[int]] = {}

    for i, doc in enumerate(documents):
        words = doc.split()
        for word in words:
            if word not in inverted_index:
                inverted_index[word] = []
            if i not in inverted_index[word]:   # Tránh thêm trùng ID
                inverted_index[word].append(i)

    # In ra chỉ mục để kiểm tra
    print("--- Chỉ mục ngược đã xây dựng ---")
    for word, doc_ids in inverted_index.items():
        print(f"'{word}': {doc_ids}")

    # Chức năng tìm kiếm
    search_term = input("\nNhập từ cần tìm trong tài liệu: ")
    doc_ids = inverted_index.get(search_term)
    if doc_ids is not None:
        print(f"Từ '{search_term}' xuất hiện trong các tài liệu: {doc_ids}")
    else:
        print(f"Không tìm thấy từ '{search_term}' trong bất kỳ tài liệu nào.")


if __name__ == "__main__":
    main()
```

Vận dụng AI để gia tăng trải nghiệm

- **Mở rộng cấu trúc:** Hỏi AI: "Làm thế nào để tôi nâng cấp chỉ mục này để nó không chỉ lưu ID tài liệu, mà còn lưu cả số lần từ đó xuất hiện trong mỗi tài liệu?" (AI sẽ gợi ý đổi cấu trúc thành `dict[str, dict[int, int]]`, có thể kết hợp `collections.defaultdict`).
- **Tối ưu hóa:** Hỏi AI: "Nếu tập tài liệu rất lớn, việc xây dựng chỉ mục này có thể tốn bộ nhớ. Hãy gợi ý một vài kỹ thuật để tối ưu hóa bộ nhớ cho chỉ mục ngược trong Python."

BÀI TẬP 6: Cặp số có tổng bằng K (Two Sum - Không sắp xếp) - Mức độ: Dễ - Trung bình (Dictionary)

Bài toán phỏng vấn kinh điển (Two Sum) nhưng giải quyết bằng Bảng băm (Dictionary) để đạt độ phức tạp O(n) trên mảng chưa được sắp xếp.

Yêu cầu chi tiết:

- Cho một mảng số nguyên nums và một số nguyên target.
- Tìm và trả về chỉ số (index) của 2 số sao cho tổng của chúng bằng đúng target.
- Giả định mỗi bài toán chỉ có đúng một đáp án và không được dùng một phần tử hai lần.

💡 **Gợi ý:**

- Khởi tạo một Dictionary rỗng seen để lưu lịch sử dưới dạng: Key = Giá trị con số, Value = Vị trí (index) của số đó.
- Duyệt qua từng số trong mảng, tính phần bù cần tìm: complement = target - num.
- Kiểm tra xem complement đã có trong Dictionary seen chưa. Nếu có, ta đã tìm thấy cặp số. Nếu chưa, lưu số hiện tại và vị trí của nó vào seen để các bước sau có thể tìm thấy.

💻 **Code mẫu (Python):**

```
def two_sum(nums: list[int], target: int) -> list[int]:
    seen = {}  # Lưu trữ giá trị: vị trí

    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i

    return []

def main():
    nums = [2, 7, 11, 15]
    target = 9
    result = two_sum(nums, target)
    print(f"Mảng: {nums}, Mục tiêu: {target}")
    print(f"Chỉ số tìm thấy: {result} (Vì {nums[result[0]]} + {nums[result[1]]} =
{target})")

if __name__ == "__main__":
    main()
```

🤖 **Vận dụng AI để gia tăng trải nghiệm:**

- **Phân tích thuật toán:** Hỏi AI: *"So sánh chi tiết về độ phức tạp thời gian (Time Complexity) và không gian (Space Complexity) giữa cách giải Two Sum bằng 2 vòng lặp lồng nhau (O(n^2)) và cách giải bằng Dictionary (O(n))."*

BÀI TẬP 7: Gom nhóm các từ đảo chữ (Group Anagrams) - Mức độ: Trung bình (Dictionary)

Bài tập rèn luyện kỹ năng sử dụng các cấu trúc dữ liệu lồng nhau và hiểu tính chất của Key trong Dictionary.

Yêu cầu chi tiết:

- Cho một mảng các chuỗi. Hãy gom nhóm các từ là "đảo chữ" của nhau (Anagram - chứa cùng số lượng các ký tự giống nhau nhưng sắp xếp khác vị trí).
- *Ví dụ:* ["eat", "tea", "tan", "ate", "nat", "bat"] -> Kết quả: [["bat"], ["nat", "tan"], ["ate", "eat", "tea"]].

💡 **Gợi ý:**

- Có thể dùng collections.defaultdict(list) để tránh việc phải kiểm tra Key đã tồn tại hay chưa.
- Nếu hai từ là đảo chữ của nhau, khi sắp xếp các ký tự theo bảng chữ cái, chúng sẽ biến thành một chuỗi giống hệt nhau (ví dụ: "eat" và "tea" đều thành "aet").
- Sử dụng chuỗi đã sắp xếp này làm Key, Value là một danh sách chứa các từ gốc.

💻 **Code mẫu (Python):**

```
from collections import defaultdict

def group_anagrams(strs: list[str]) -> list[list[str]]:
    # Dùng defaultdict(list) để khởi tạo sẵn danh sách rỗng cho key mới
    anagram_groups = defaultdict(list)

    for word in strs:
        # Sắp xếp các ký tự trong từ để tạo ra một chuỗi "chữ ký" duy nhất
        # sorted() trả về list, cần ''.join() để nối lại thành chuỗi (string)
        signature = ''.join(sorted(word))

        # Thêm từ gốc vào nhóm có cùng chữ ký
        anagram_groups[signature].append(word)

    return list(anagram_groups.values())

def main():
    words = ["eat", "tea", "tan", "ate", "nat", "bat"]
    result = group_anagrams(words)
    print("Mảng gốc:", words)
    print("Các nhóm đảo chữ:")
    for group in result:
        print(f" - {group}")

if __name__ == "__main__":
    main()
```

Vận dụng AI để gia tăng trải nghiệm:

- **Hiểu bản chất ngôn ngữ:** Hỏi AI: *"Tại sao Key của Dictionary trong Python bắt buộc phải là kiểu dữ liệu bất biến (immutable) như String hoặc Tuple? Điều gì xảy ra nếu tôi cố dùng List làm Key?"*

Bài tập 8: Từ điển đồng nghĩa đa chiều - Mức độ: Trung bình (Dictionary & Set)

Bài tập kết hợp giữa Dictionary và Set để quản lý tập hợp dữ liệu không trùng lặp.

Yêu cầu chi tiết:

- Viết chương trình quản lý từ đồng nghĩa.
- Mỗi từ có thể có nhiều từ đồng nghĩa, và không được lưu trùng lặp.
- Cung cấp các chức năng:
    1. Thêm một cặp từ đồng nghĩa (ví dụ: Thêm "vui" và "hạnh phúc"). Chú ý: Đây là quan hệ hai chiều (thêm "vui" có đồng nghĩa là "hạnh phúc" thì ngược lại, "hạnh phúc" cũng có đồng nghĩa là "vui").
    2. Tra cứu danh sách từ đồng nghĩa của một từ.

Gợi ý:

- Dùng dict[str, set[str]]. Set (Tập hợp) giúp tự động loại bỏ các từ đồng nghĩa bị chèn trùng.
- Khi thêm cặp (A, B), phải thêm B vào Set của A và thêm A vào Set của B.

Code mẫu (Python):

```
from collections import defaultdict

class Thesaurus:
    def __init__(self):
        # Value là một Set để loại bỏ trùng lặp
```

```
        self.data = defaultdict(set)

    def add_synonyms(self, word1: str, word2: str):
        self.data[word1].add(word2)
        self.data[word2].add(word1)
        print(f"Đã liên kết '{word1}' và '{word2}'")

    def lookup(self, word: str):
        if word in self.data:
            synonyms = ", ".join(self.data[word])
            print(f"Các từ đồng nghĩa với '{word}': {synonyms}")
        else:
            print(f"Không tìm thấy dữ liệu cho từ '{word}'")

def main():
    dictionary = Thesaurus()

    dictionary.add_synonyms("vui", "hạnh phúc")
    dictionary.add_synonyms("vui", "phấn khởi")
    dictionary.add_synonyms("hạnh phúc", "sung sướng")
    dictionary.add_synonyms("vui", "hạnh phúc") # Cố tình thêm trùng

    print("-" * 30)
    dictionary.lookup("vui")
    dictionary.lookup("hạnh phúc")
    dictionary.lookup("buồn")

if __name__ == "__main__":
    main()
```

Vận dụng AI để gia tăng trải nghiệm:

- **So sánh cấu trúc:** Hỏi AI: *"Sự khác biệt cốt lõi về cấu trúc dữ liệu giữa Set và List trong Python là gì? Tại sao việc kiểm tra một phần tử có tồn tại trong Set (in) lại nhanh hơn rất nhiều so với List?"*

Bài tập 9: Kiểm tra cửa sổ trùng lặp (Contains Duplicate II) - Mức độ: Khó (Sliding Window & Dictionary)

Ứng dụng Bảng băm để theo dõi khoảng cách vị trí của các phần tử.

Yêu cầu chi tiết:

- Cho một mảng số nguyên nums và một số nguyên k.
- Trả về True nếu có hai chỉ số i và j khác nhau trong mảng sao cho nums[i] == nums[j] và khoảng cách abs(i - j) <= k. Ngược lại trả về False.
- *Ví dụ:* nums = [1, 2, 3, 1], k = 3 -> True.
- *Ví dụ:* nums = [1, 2, 3, 1, 2, 3], k = 2 -> False.

Gợi ý:

- Dùng Dictionary recent_indices để lưu trữ vị trí (index) xuất hiện *gần nhất* của mỗi con số.
- Khi duyệt qua mảng, nếu số hiện tại đã có trong Dictionary, ta kiểm tra khoảng cách: vị_trí_hiện_tại - recent_indices[num] <= k.
- Nếu đúng, trả về True. Nếu sai, cập nhật lại recent_indices[num] = vị_trí_hiện_tại.

Code mẫu (Python):

```
def contains_nearby_duplicate(nums: list[int], k: int) -> bool:
    recent_indices = {}

    for i, num in enumerate(nums):
        if num in recent_indices:
            # Kiểm tra khoảng cách
            if i - recent_indices[num] <= k:
                return True

        # Cập nhật vị trí xuất hiện mới nhất
        recent_indices[num] = i

    return False

def main():
    test_cases = [
        ([1, 2, 3, 1], 3),
        ([1, 0, 1, 1], 1),
        ([1, 2, 3, 1, 2, 3], 2)
    ]

    for nums, k in test_cases:
        result = contains_nearby_duplicate(nums, k)
        print(f"Mảng: {nums}, k = {k} -> Kết quả: {result}")

if __name__ == "__main__":
    main()
```

Vận dụng AI để gia tăng trải nghiệm:

- **Tìm hiểu thuật toán:** Hỏi AI: *"Khái niệm Cửa sổ trượt (Sliding Window) là gì trong khoa học máy tính? Bài toán này có thể được giải bằng Set thay vì Dictionary để duy trì một cửa sổ có kích thước K được không? Viết thử code minh họa."*

Bài tập 10: Hệ thống bình chọn trực tuyến (Voting System) - Mức độ: Rất khó (Tổng hợp Dictionary & Sorting)

Bài tập thiết kế hệ thống tổng hợp, đòi hỏi sử dụng nhiều Dictionary cùng lúc để đảm bảo tính toàn vẹn của dữ liệu và truy xuất Top kết quả nhanh.

Yêu cầu chi tiết:

- Xây dựng hệ thống bình chọn cho cuộc thi.
- Điều kiện: Mỗi người dùng (UserID) chỉ được phép bình chọn tối đa 1 lần cho 1 ứng cử viên (Candidate). Nếu cố tình bình chọn lần 2 sẽ bị từ chối.
- Cung cấp các chức năng:
  1. Ghi nhận lượt bình chọn (Vote).
  2. Lấy ra danh sách Top 3 ứng cử viên có số phiếu cao nhất (xếp hạng giảm dần). Nếu bằng phiếu nhau, xếp theo tên bảng chữ cái.

Gợi ý:

- Dùng has_voted: dict[str, bool] để kiểm tra nhanh (O(1)) xem UserID đã bầu cử chưa.
- Dùng vote_counts: dict[str, int] để lưu tổng số phiếu của từng Candidate.

- Để lấy Top 3, sử dụng hàm sorted() của Python áp dụng lên vote_counts.items(), với key sắp xếp ưu tiên giảm dần theo phiếu, sau đó tăng dần theo tên.

💻 **Code mẫu (Python):**

```
class VotingSystem:
    def __init__(self):
        self.has_voted = {}    # Lưu trữ: { user_id: bool }
        self.vote_counts = {}  # Lưu trữ: { candidate_name: int }

    def vote(self, user_id: str, candidate: str):
        if user_id in self.has_voted:
            print(f"[Từ chối] User '{user_id}' đã bình chọn rồi.")
            return False

        self.has_voted[user_id] = True
        self.vote_counts[candidate] = self.vote_counts.get(candidate, 0) + 1
        print(f"[Thành công] User '{user_id}' đã bình chọn cho '{candidate}'.")
        return True

    def get_top_3(self):
        # Sắp xếp danh sách tuples (candidate, votes)
        # key = lambda x: (-x[1], x[0]) nghĩa là:
        # Tiêu chí 1: -x[1] (Sắp xếp số phiếu giảm dần)
        # Tiêu chí 2: x[0] (Nếu bằng phiếu, sắp xếp tên A-Z)
        sorted_candidates = sorted(self.vote_counts.items(), key=lambda x: (-x[1],
x[0]))

        print("\n--- TOP 3 ỨNG CỬ VIÊN ---")
        for i, (candidate, votes) in enumerate(sorted_candidates[:3], start=1):
            print(f"Hạng {i}: {candidate} ({votes} phiếu)")

def main():
    system = VotingSystem()

    # Kịch bản bình chọn
    system.vote("user_001", "Alice")
    system.vote("user_002", "Bob")
    system.vote("user_003", "Alice")
    system.vote("user_004", "Charlie")
    system.vote("user_001", "David") # Bị từ chối
    system.vote("user_005", "Charlie")
    system.vote("user_006", "Alice")
    system.vote("user_007", "Bob")

    # Lấy top 3
    system.get_top_3()

if __name__ == "__main__":
    main()
```

🤖 **Vận dụng AI để gia tăng trải nghiệm:**

- **Tối ưu hệ thống lớn:** Hỏi AI: *"Nếu số lượng ứng cử viên lên đến 10 triệu người, việc dùng sorted() cho toàn bộ Dictionary để lấy Top 3 sẽ mất O(N \log N) và tốn nhiều RAM. Hãy hướng dẫn cách dùng cấu trúc dữ liệu Hàng đợi ưu tiên (Priority Queue / module heapq.nlargest) trong Python để lấy Top 3 với độ phức tạp tối ưu hơn."*